

# **Лабораторная работа №3**

**Управляющие структуры**

Александр Эдуардович Аскеров

# Содержание

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                           | <b>6</b>  |
| <b>2</b> | <b>Задание</b>                               | <b>7</b>  |
| <b>3</b> | <b>Выполнение лабораторной работы</b>        | <b>8</b>  |
| 3.1      | Повтор примеров . . . . .                    | 8         |
| 3.2      | Задания для самостоятельной работы . . . . . | 18        |
| <b>4</b> | <b>Выводы</b>                                | <b>31</b> |
|          | <b>Список литературы</b>                     | <b>32</b> |

# Список иллюстраций

|      |                                     |    |
|------|-------------------------------------|----|
| 3.1  | Циклы 1 . . . . .                   | 8  |
| 3.2  | Циклы 2 . . . . .                   | 9  |
| 3.3  | Циклы 3 . . . . .                   | 9  |
| 3.4  | Циклы 4 . . . . .                   | 10 |
| 3.5  | Циклы 5 . . . . .                   | 10 |
| 3.6  | Циклы 6 . . . . .                   | 11 |
| 3.7  | Условные выражения 1 . . . . .      | 11 |
| 3.8  | Условные выражения 2 . . . . .      | 12 |
| 3.9  | Функции 1 . . . . .                 | 12 |
| 3.10 | Функции 2 . . . . .                 | 12 |
| 3.11 | Функции 3 . . . . .                 | 13 |
| 3.12 | Функции 4 . . . . .                 | 13 |
| 3.13 | Функции 5 . . . . .                 | 13 |
| 3.14 | Функции 6 . . . . .                 | 14 |
| 3.15 | Функции 7 . . . . .                 | 14 |
| 3.16 | Функции 8 . . . . .                 | 15 |
| 3.17 | Функции 9 . . . . .                 | 16 |
| 3.18 | Функции 10 . . . . .                | 16 |
| 3.19 | Функции 11 . . . . .                | 17 |
| 3.20 | Пакеты 1 . . . . .                  | 17 |
| 3.21 | Пакеты 2 . . . . .                  | 18 |
| 3.22 | Пакеты 3 . . . . .                  | 18 |
| 3.23 | Задание 1.1 . . . . .               | 19 |
| 3.24 | Задание 1.2 . . . . .               | 19 |
| 3.25 | Задание 1.3 . . . . .               | 19 |
| 3.26 | Задание 2 . . . . .                 | 20 |
| 3.27 | Задание 3 . . . . .                 | 20 |
| 3.28 | Задание 4 . . . . .                 | 21 |
| 3.29 | Задание 5.1 . . . . .               | 21 |
| 3.30 | Задание 5.2 . . . . .               | 22 |
| 3.31 | Задание 6 . . . . .                 | 23 |
| 3.32 | Задание 7 . . . . .                 | 24 |
| 3.33 | Задание 7 (продолжение) . . . . .   | 25 |
| 3.34 | Задание 8.1 . . . . .               | 26 |
| 3.35 | Задание 8.2 . . . . .               | 27 |
| 3.36 | Задание 8.2 (продолжение) . . . . . | 28 |

|      |                        |    |
|------|------------------------|----|
| 3.37 | Задание 9 . . . . .    | 29 |
| 3.38 | Задание 10.1 . . . . . | 30 |
| 3.39 | Задание 10.2 . . . . . | 30 |
| 3.40 | Задание 10.3 . . . . . | 30 |
| 3.41 | Задание 11 . . . . .   | 30 |

## **Список таблиц**

# 1 Цель работы

Основная цель работы – освоить применение циклов, функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

## 2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 3.2.
2. Выполните задания для самостоятельной работы (раздел 3.4).

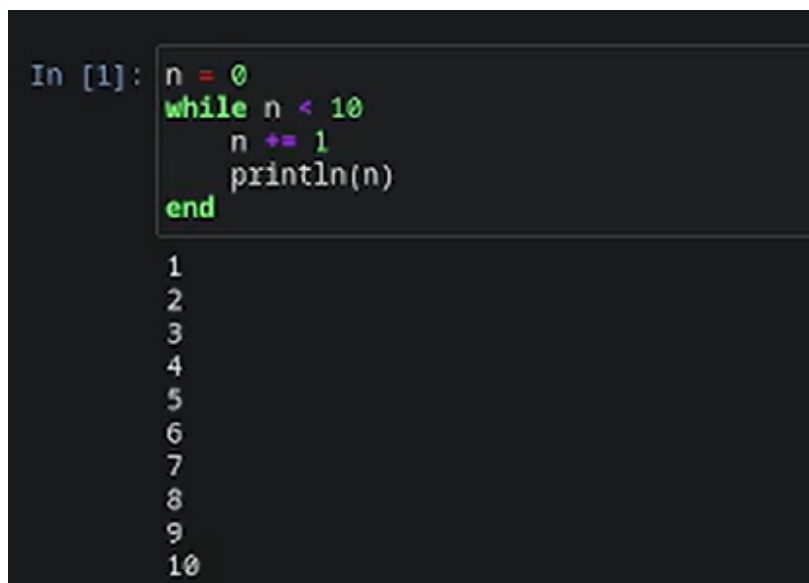
## 3 Выполнение лабораторной работы

### 3.1 Повтор примеров

Выполним примеры из раздела циклы while и for [1].

Для различных операций, связанных с перебором индексируемых элементов структур данных, традиционно используются циклы while и for.

Например, while можно использовать для формирования элементов массива:



```
In [1]: n = 0
while n < 10
    n += 1
    println(n)
end

1
2
3
4
5
6
7
8
9
10
```

Рисунок 3.1: Циклы 1

Другой пример демонстрирует использование while при работе со строковыми элементами массива, подставляя имя из массива в заданную строку приветствия и выводя получившуюся конструкцию на экран:

```

myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
i = 1
while i <= length(myfriends)
    friend = myfriends[i]
    println("Hi $friend, it's great to see you!")
    i += 1
end

Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!

```

Рисунок 3.2: Циклы 2

Такие же результаты можно получить при использовании цикла for.

Рассмотренные выше примеры, но с использованием цикла for:

```

for n in 1:2:10
    println(n)
end
myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
for friend in myfriends
    println("Hi $friend, it's great to see you!")
end

1
3
5
7
9
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!

```

Рисунок 3.3: Циклы 3

Пример использования цикла for для создания двумерного массива, в котором значение каждой записи является суммой индексов строки и столбца:

```

m, n = 5, 5
A = fill{0, (m, n)}

for i in 1:m
    for j in 1:n
        A[i, j] = i + j
    end
end
A

```

5×5 Matrix{Int64}:

|   |   |   |   |    |
|---|---|---|---|----|
| 2 | 3 | 4 | 5 | 6  |
| 3 | 4 | 5 | 6 | 7  |
| 4 | 5 | 6 | 7 | 8  |
| 5 | 6 | 7 | 8 | 9  |
| 6 | 7 | 8 | 9 | 10 |

Рисунок 3.4: Циклы 4

Другая реализация этого же примера:

```

B = fill{0, (m, n)}
for i in 1:m, j in 1:n
    B[i, j] = i + j
end
B

```

5×5 Matrix{Int64}:

|   |   |   |   |    |
|---|---|---|---|----|
| 2 | 3 | 4 | 5 | 6  |
| 3 | 4 | 5 | 6 | 7  |
| 4 | 5 | 6 | 7 | 8  |
| 5 | 6 | 7 | 8 | 9  |
| 6 | 7 | 8 | 9 | 10 |

Рисунок 3.5: Циклы 5

Ещё одна реализация этого же примера:

```

C = [i + j for i in 1:m, j in 1:n]
C
5×5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10

```

Рисунок 3.6: Циклы 6

Выполним примеры из раздела условные выражения.

Довольно часто при решении задач требуется проверить выполнение тех или иных условий. Для этого используют условные выражения.

Например, пусть для заданного числа  $N$  требуется вывести слово «Fizz», если  $N$  делится на 3, «Buzz», если  $N$  делится на 5, и «FizzBuzz», если  $N$  делится на 3 и 5:

```

N = 10
if (N % 3 == 0) && (N % 5 == 0)
    println("FizzBuzz")
elseif N % 3 == 0
    println("Fizz")
elseif N % 5 == 0
    println("Buzz")
else
    println(N)
end
Buzz

```

Рисунок 3.7: Условные выражения 1

Пример использования тернарного оператора:

```
x = 5
y = 10
(x > y) ? x : y

10
```

Рисунок 3.8: Условные выражения 2

Выполним примеры из раздела функции.

Julia дает нам несколько разных способов написать функцию. Первый требует ключевых слов `function` и `end`:

```
function sayhi(name)
    println("Hi $name, it's great to see you!")
end

function f(x)
    x^2
end
```

Рисунок 3.9: Функции 1

Вызов функции осуществляется по её имени с указанием аргументов, например:

```
sayhi("С-ЗРО")
f(42)

Hi С-ЗРО, it's great to see you!
1764
```

Рисунок 3.10: Функции 2

В качестве альтернативы, можно объявить любую из выше определённых функций в одной строке:

```
sayhi2(name) = println("Hi $name, it's great to see you!")
f2(x) = x^2
f2 (generic function with 1 method)
```

Рисунок 3.11: Функции 3

Наконец, можно объявить выше определённые функции как «анонимные»:

```
sayhi3 = name -> println("Hi $name, it's great to see you!")
f3 = x -> x^2
#5 (generic function with 1 method)
```

Рисунок 3.12: Функции 4

По соглашению в Julia функции, сопровождаемые восклицательным знаком, изменяют свое содержимое, а функции без восклицательного знака не делают этого.

Например, сравните результат применения `sort` и `sort!`:

```
v = [3, 5, 2]
sort(v)
print(v)
sort!(v)
print(v)

[3, 5, 2] [2, 3, 5]
```

Рисунок 3.13: Функции 5

Функция `sort(v)` возвращает отсортированный массив, который содержит те же элементы, что и массив `v`, но исходный массив `v` остаётся без изменений. Если же использовать `sort!(v)`, то отсортировано будет содержимое исходного массива `v`.

В программировании под функцией высшего порядка понимается функция, принимающая в качестве аргументов другие функции или возвращающая другую

функцию в качестве результата. Основная идея состоит в том, что функции имеют тот же статус, что и другие объекты данных.

В Julia функция `map` является функцией высшего порядка, которая принимает функцию в качестве одного из своих входных аргументов и применяет эту функцию к каждому элементу структуры данных, которая ей передаётся также в качестве аргумента.

```
f(x) = x^2
map(f, [1, 2, 3])

3-element Vector{Int64}:
 1
 4
 9
```

Рисунок 3.14: Функции 6

То есть в квадрат возведены все элементы массива `[1, 2, 3]`, но не сам массив (вектор).

В `map` можно передать и анонимно заданную, а не именованную функцию:

```
map(x -> x^3, [1, 2, 3])

3-element Vector{Int64}:
 1
 8
27
```

Рисунок 3.15: Функции 7

Функция `broadcast` – ещё одна функция высшего порядка в Julia, представляющая собой обобщение функции `map`. Функция `broadcast()` будет пытаться привести все объекты к общему измерению, `map()` будет напрямую применять данную функцию поэлементно.

Синтаксис для вызова `broadcast` такой же, как и для вызова `map`, например применение функции `f` к элементам массива `[1, 2, 3]`:

```
f(x) = x^2
broadcast(f, [1, 2, 3])

3-element Vector{Int64}:
 1
 4
 9

f.([1, 2, 3])

3-element Vector{Int64}:
 1
 4
 9
```

Рисунок 3.16: Функции 8

Пример:

```

A = [i + 3*j for j in 0:2, i in 1:3]

3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9

f(A)

3×3 Matrix{Int64}:
30  36  42
66  81  96
102 126 150

```

Рисунок 3.17: Функции 9

С другой стороны,

```

B = f.(A)

3×3 Matrix{Int64}:
 1  4  9
16 25 36
49 64 81

```

Рисунок 3.18: Функции 10

Точечный синтаксис для `broadcast()` позволяет записать относительно сложные составные поэлементные выражения в форме, близкой к математической записи.

```

A .* 2 .* f.(A) ./ A

3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

julia> A + 2 * f(A) / A

3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

broadcast(x -> x + 2 * f(x) / x, A)

3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

```

Рисунок 3.19: Функции 11

Выполним примеры из раздела сторонние библиотеки (пакеты) в Julia.

При первом использовании пакета в вашей текущей установке Julia вам необходимо использовать менеджер пакетов, чтобы явно его добавить.

При каждом новом использовании Julia (например, в начале нового сеанса в REPL или открытии блокнота в первый раз) нужно загрузить пакет, используя ключевое слово `using`.

Например, добавим и загрузим пакет `Colors`:

```

import Pkg
Pkg.add("Colors")

```

Рисунок 3.20: Пакеты 1

Затем создадим палитру из 100 разных цветов:

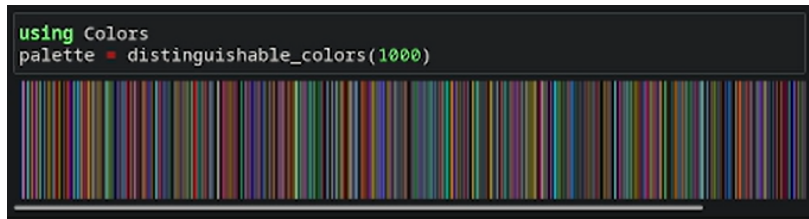


Рисунок 3.21: Пакеты 2

А затем определим матрицу  $3 \times 3$  с элементами в форме случайного цвета из палитры, используя функцию `rand`:



Рисунок 3.22: Пакеты 3

## 3.2 Задания для самостоятельной работы

1. Используя циклы `while` и `for`:

1.1. Выведите на экран целые числа от 1 до 100 и напечатайте их квадраты.

```
for i in 1:100
    print(i, ":", i^2, ";;")
end

1: 1;; 2: 4;; 3: 9;; 4: 16;; 5: 25;; 6: 36;; 7: 49;; 8: 64;; 9:
81;; 10: 100;; 11: 121;; 12: 144;; 13: 169;; 14: 196;; 15: 225;;
16: 256;; 17: 289;; 18: 324;; 19: 361;; 20: 400;; 21: 441;; 22: 4
84;; 23: 529;; 24: 576;; 25: 625;; 26: 676;; 27: 729;; 28: 784;;
29: 841;; 30: 900;; 31: 961;; 32: 1024;; 33: 1089;; 34: 1156;; 35
: 1225;; 36: 1296;; 37: 1369;; 38: 1444;; 39: 1521;; 40: 1600;; 4
1: 1681;; 42: 1764;; 43: 1849;; 44: 1936;; 45: 2025;; 46: 2116;;
47: 2209;; 48: 2304;; 49: 2401;; 50: 2500;; 51: 2601;; 52: 2704;;
53: 2809;; 54: 2916;; 55: 3025;; 56: 3136;; 57: 3249;; 58: 3364;;
59: 3481;; 60: 3600;; 61: 3721;; 62: 3844;; 63: 3969;; 64: 4096;;
65: 4225;; 66: 4356;; 67: 4489;; 68: 4624;; 69: 4761;; 70: 4900;;
71: 5041;; 72: 5184;; 73: 5329;; 74: 5476;; 75: 5625;; 76: 5776;;
77: 5929;; 78: 6084;; 79: 6241;; 80: 6400;; 81: 6561;; 82: 6724;;
83: 6889;; 84: 7056;; 85: 7225;; 86: 7396;; 87: 7569;; 88: 7744;;
89: 7921;; 90: 8100;; 91: 8281;; 92: 8464;; 93: 8649;; 94: 8836;;
95: 9025;; 96: 9216;; 97: 9409;; 98: 9604;; 99: 9801;; 100: 10000;
;;
```

Рисунок 3.23: Задание 1.1

1.2. Создайте словарь squares, который будет содержать целые числа в качестве ключей и квадраты в качестве их пар-значений.

```
squares = Dict{<
for i in 1:5
    squares[i] = i^2
end
print(squares)

Dict{Any, Any}(5 => 25, 4 => 16, 2 => 4, 3 => 9, 1 => 1)
```

Рисунок 3.24: Задание 1.2

1.3. Создайте массив squares\_arr, содержащий квадраты всех чисел от 1 до 100.

```
squares_arr = [i^2 for i in 1:100]
print(squares_arr)

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 28
9, 324, 361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 961
, 1024, 1089, 1156, 1225, 1296, 1369, 1444, 1521, 1600, 1681, 1764, 184
9, 1936, 2025, 2116, 2209, 2304, 2401, 2500, 2601, 2704, 2809, 2916, 30
25, 3136, 3249, 3364, 3481, 3600, 3721, 3844, 3969, 4096, 4225, 4356, 4
489, 4624, 4761, 4900, 5041, 5184, 5329, 5476, 5625, 5776, 5929, 6084,
6241, 6400, 6561, 6724, 6889, 7056, 7225, 7396, 7569, 7744, 7921, 8100,
8281, 8464, 8649, 8836, 9025, 9216, 9409, 9604, 9801, 10000]
```

Рисунок 3.25: Задание 1.3

2. Напишите условный оператор, который печатает число, если число чётное, и строку «нечётное», если число нечётное. Перепишите код, используя тернарный оператор.

```
x = 9
if x % 2 == 0
    println(x)
elseif x % 2 != 0
    println("odd")
end

odd

(x % 2 == 0) ? println(x) : println("odd")

odd
```

Рисунок 3.26: Задание 2

3. Напишите функцию `add_one`, которая добавляет 1 к своему входу.

```
function add_one(x)
    x += 1
    return x
end

add_one!(2)

3
```

Рисунок 3.27: Задание 3

4. Используйте `map()` или `broadcast()` для задания матрицы  $A$ , каждый элемент которой увеличивается на единицу по сравнению с предыдущим.

```

A = fill(0, (3, 3))
broadcast(x -> x+1, A)

3×3 Matrix{Int64}:
 1  1  1
 1  1  1
 1  1  1

```

Рисунок 3.28: Задание 4

5. Задайте матрицу  $A$ :

5.1. Найдите  $A^3$ .

```

A = [1 1 3; 5 2 6; -2 -1 -3]

3×3 Matrix{Int64}:
 1  1  3
 5  2  6
-2 -1 -3

f(x) = x^3
f.(A)

3×3 Matrix{Int64}:
 1  1  27
125  8  216
-8 -1 -27

```

Рисунок 3.29: Задание 5.1

5.2. Замените третий столбец матрицы  $A$  на сумму второго и третьего столбцов.

```
for i in 1:3
    A[i, 3] = A[i, 1] + A[i, 2]
end
A
```

```
3×3 Matrix{Int64}:
 1  1  2
 5  2  7
-2 -1 -3
```

Рисунок 3.30: Задание 5.2

6. Создание матрицы  $B$ . Вычисление матрицы  $C$ .

```
B = zeros(15, 3)
for i in 1:15
    B[i, 1] = 10
    B[i, 2] = -10
    B[i, 3] = 10
end
B

15×3 Matrix{Float64}:
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0
 10.0  -10.0  10.0

C = B' * B

3×3 Matrix{Float64}:
 1500.0  -1500.0  1500.0
-1500.0   1500.0 -1500.0
 1500.0  -1500.0  1500.0
```

Рисунок 3.31: Задание 6

7. Создание матриц  $Z_1, Z_2, Z_3, Z_4$ .

```

Z = fill(0, (6, 6))
E = fill(1, (6, 6))
Z1, Z2, Z3, Z4 = copy(Z), copy(Z), copy(Z), copy(Z)
for i in 1:6, j in 1:6
    if j == i - 1 || j == i + 1
        Z1[i, j] = 1
    end
end
for i in 1:6, j in 1:6
    if j == i - 2 || j == i || j == i + 2
        Z2[i, j] = 1
    end
end
for i in 1:6, j in 1:6
    if j == 5 - i || j == 7 - i || j == 9 - i
        Z3[i, j] = 1
    end
end
for i in 1:6, j in 1:6
    if (i + j) % 2 == 0
        Z4[i, j] = 1
    end
end
Z1

6x6 Matrix{Int64}:
 0  1  0  0  0  0
 1  0  1  0  0  0
 0  1  0  1  0  0
 0  0  1  0  1  0
 0  0  0  1  0  1
 0  0  0  0  1  0

```

Рисунок 3.32: Задание 7

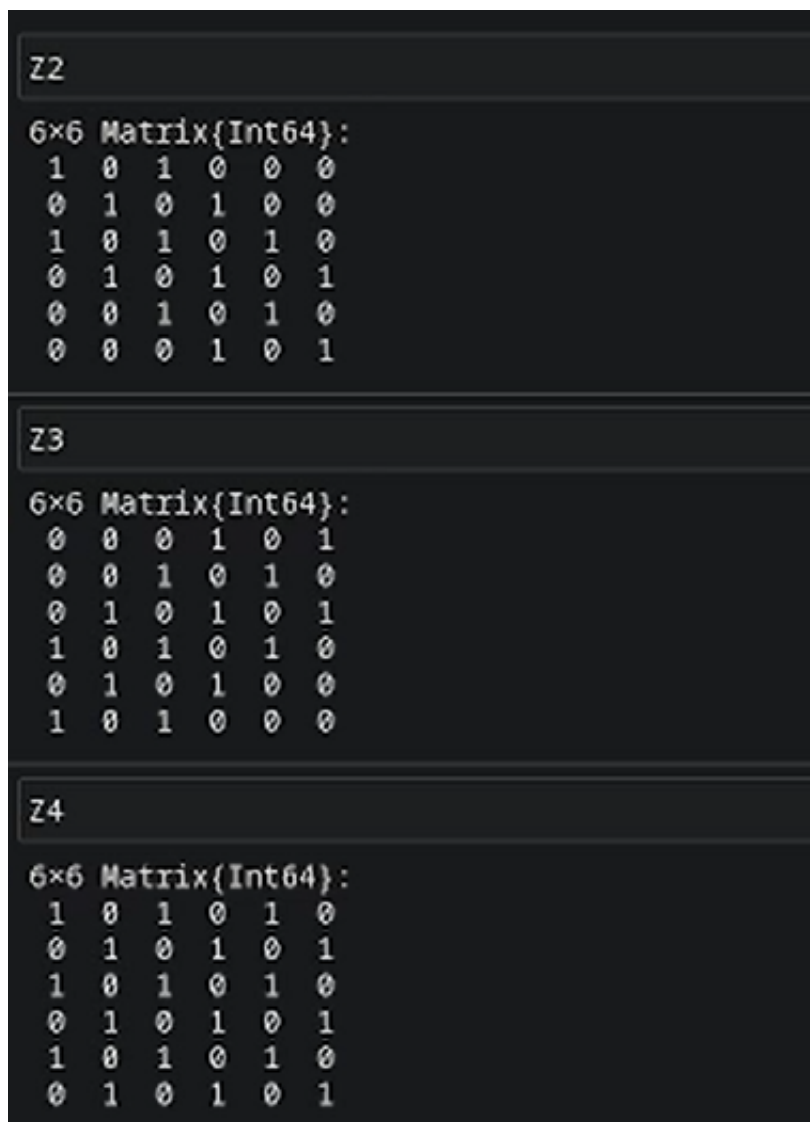


Рисунок 3.33: Задание 7 (продолжение)

8. В языке R есть функция `outer()`. Фактически, это матричное умножение с возможностью изменить применяемую операцию (например, заменить произведение на сложение или возведение в степень).

8.1. Напишите свою функцию, аналогичную функции `outer()` языка R.

```

function outer(x, y, f)
    if (f == "*")
        res = x .* y
    elseif (f == "+")
        res = x .+ y
    elseif (f == "^")
        res = x .^ y
    elseif (f == "-")
        res = x .- y
    elseif (f == "%")
        res = x ./ y
    end
end

```

Рисунок 3.34: Задание 8.1

8.2. Используя написанную вами функцию `outer()`, создайте матрицы  $A_1$ ,  $A_2$ ,  $A_3$ ,  $A_4$ ,  $A_5$ .

```

A1 = outer(0:4, 0:4, "+")
A1

5×5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  5
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8

A2 = outer(0:4, 1:5, "^")
A2

5×5 Matrix{Int64}:
 0  0  0  0  0
 1  1  1  1  1
 2  4  8  16  32
 3  9  27  81  243
 4  16  64  256  1024

A3 = outer(outer(0:4, 0:4, "+"), 5, "%")
A3

5×5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  0
 2  3  4  0  1
 3  4  0  1  2
 4  0  1  2  3

```

Рисунок 3.35: Задание 8.2

```
A4 = outer(outer(0:9, 0:9, "+"), 10, "%")
10x10 Matrix{Int64}:
 0  1  2  3  4  5  6  7  8  9
 1  2  3  4  5  6  7  8  9  0
 2  3  4  5  6  7  8  9  0  1
 3  4  5  6  7  8  9  0  1  2
 4  5  6  7  8  9  0  1  2  3
 5  6  7  8  9  0  1  2  3  4
 6  7  8  9  0  1  2  3  4  5
 7  8  9  0  1  2  3  4  5  6
 8  9  0  1  2  3  4  5  6  7
 9  0  1  2  3  4  5  6  7  8

A5 = outer(1:8, 8:-1:1, "+")
A5 = outer(A5, 9, "%")
8x8 Matrix{Int64}:
 0  8  7  6  5  4  3  2
 1  0  8  7  6  5  4  3
 2  1  0  8  7  6  5  4
 3  2  1  0  8  7  6  5
 4  3  2  1  0  8  7  6
 5  4  3  2  1  0  8  7
 6  5  4  3  2  1  0  8
 7  6  5  4  3  2  1  0
```

Рисунок 3.36: Задание 8.2 (продолжение)

9. Решите систему линейных уравнений с 5 неизвестными.

```
A = [
    1 2 3 4 5;
    2 1 2 3 4;
    3 2 1 2 3;
    4 3 2 1 2;
    5 4 3 2 1
]
y = [7; -1; -3; 5; 17]
x = A \ y

5-element Vector{Float64}:
-2.00000000000000036
 3.00000000000000058
 4.9999999999999998
 1.9999999999999991
-3.9999999999999999
```

Рисунок 3.37: Задание 9

10. Создайте матрицу  $M$  размерности  $6 \times 10$ , элементами которой являются целые числа, выбранные случайным образом с повторениями из совокупности  $1, 2, \dots, 10$ .

10.1. Найдите число элементов в каждой строке матрицы  $M$ , которые больше числа  $N$  (например,  $N = 4$ ).

```

M = rand(1:10, 6, 10)
println(M)
N = 4
sum(M.>N)

[7 7 3 3 1 8 8 3 8 4; 4 9 8 2 6 8 3 5 2 1; 5 3 2 4 7 5 3 4 3 10; 5 5 9
5 2 8 10 9 4 10; 3 3 3 3 9 7 2 1 5 7; 7 9 9 8 2 9 5 5 9 9]

35

```

Рисунок 3.38: Задание 10.1

10.2. Определите, в каких строках матрицы  $M$  число  $M$  (например,  $M = 7$ ) встречается ровно 2 раза?

```

[i for i in 1:6 if sum(M[i, :] .== 7) == 2]

2-element Vector{Int64}:
 1
 5

```

Рисунок 3.39: Задание 10.2

10.3. Определите все пары столбцов матрицы  $M$ , сумма элементов которых больше  $K$  (например,  $K = 75$ ).

```

K = 75
[(i, j) for i in 1:6, j in 2:5 if (i != j && sum(M[:, i] + M[:, j]) > K)]

2-element Vector{Tuple{Int64, Int64}}:
 (6, 2)
 (6, 3)

```

Рисунок 3.40: Задание 10.3

11. Вычислите суммы.

```

sum1 = sum((i^4)/(3 + j) for i in 1:20, j in 1:5)

639215.2833333338

sum2 = sum((i^4)/(3 + i * j) for i in 1:20, j in 1:5)

89912.02146097131

```

Рисунок 3.41: Задание 11

## 4 Выводы

Освоено применение циклов, функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

# Список литературы

- [1] Julia Language Documentation. *Julia 1.5 Documentation*. 2020. URL: <https://docs.julialang.org/en/v1/>.